

Pairwise Sequence Alignment Algorithms – A Survey

Waqar Haque, Alex Aravind, Bharath Reddy

Computer Science Program, University of Northern British Columbia

Prince George, BC, Canada V2N4Z9

haque@unbc.ca, csalex@unbc.ca, govinda@unbc.ca

ABSTRACT

Pairwise sequence alignment is a fundamental compute-intensive problem in bioinformatics that has helped researchers analyse biological sequences. The analysis has helped biologists detect pathogens, develop drugs, and identify common genes. The biological sequence database has been growing rapidly due to new sequences being discovered. This has brought many new challenges including sequence database searching and aligning long sequences. To solve these problems, many sequence alignment algorithms have been developed. These algorithms employ various techniques to efficiently find optimal or nearly-optimal alignments. In this paper, we present the popular past and recent work on both local and global pairwise sequence alignment algorithms. In addition to identifying the techniques used, the advantages and limitations of the algorithms are also presented.

Keywords

Sequence alignment, pairwise, seed, anchor, suffix tree, look-up table.

1. INTRODUCTION

Computational microbiology has gained significant popularity in the last few decades. The large volume of biological data that is stored in the form of DNA, RNA and protein sequences requires extensive computing power to retrieve and analyze sequences quickly and accurately. Aligning the sequences is an important and critical step in solving problems such as predicting the secondary and tertiary structure of a protein, predicting the ancestral sequence or detecting the common genes in two organisms. However, the complexity due to the sheer number of possible combinations and searches makes sequence alignment a very compute intensive problem. This complexity increases exponentially with the size of the sequences. Both hardware and software improvements are usually considered as potential directions to improve the speed and accuracy.

With new biological sequences being discovered almost on a daily basis, the biological sequence database is growing exponentially. This explosion of data demands new algorithms which are fast and yet efficient. The challenge to balance both speed and

efficiency was quickly recognised by many researchers and in the past few decades, there has been an explosion of new algorithms, of which popular algorithms are discussed in this paper. These algorithms can be categorized as optimal algorithms and heuristic algorithms. Optimal algorithms are computationally expensive but are efficient in terms of sensitivity of the alignment. Heuristic algorithms are computationally less expensive but produce sub-optimal alignment. Most of the proposed algorithms are inherently sequential and the embedded data dependency makes it difficult to parallelize the algorithms. The algorithms thus cannot fully exploit the computational power available with grid or cluster computing.

1.1 Sequence in the realm of a biologist

A sequence for a biologist is either a RNA, DNA or protein string, made of their respective alphabet set shown below:

- DNA= {A, C, G, T}
- RNA= {A, C, G, U}
- Protein= {A, R, N, D, C, Q, E, G, H, I, L, K, M, F, P, S, T, W, Y, V}

A widely accepted theory in biology is that all organisms evolved from a single cell. Over time, many organisms evolved, primarily due to some changes in their DNA sequence. This change is termed as ‘mutation’.

1.2 Sequence alignment

A sequence alignment is a way of arranging sequences of DNA, RNA and proteins with an objective to find regions of ‘similarity’, which may provide additional information on the functional, structural, evolutionary and other interests between the sequences under study. Aligned sequences are typically represented in rows, one on top of the other. For example, given two sequences, ATATAGAGGACACG and ATAGGGGACATGG, one possible alignment is shown in Figure 1. In this alignment, the vertical lines indicate the match.

```
ATATAGAGG_ACA_CG
| | | | | | |
__ATAG_GGGACATGG
```

Figure 1: An example of sequence alignment

The closely aligned regions in the aligned sequences are called ‘similar regions’. These similar regions are the regions usually conserved from previous generations. In some regions, special characters such as ‘-’, also known as *indels* are added. This insertion of a special symbol represents a mutation (change) or could be viewed as deletion from the other sequence's perspective.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ISTAT '09, March 20-22, 2009, Kuwait City, Kuwait.

Copyright 2009 ACM 978-1-60558-478-2/09/03...\$5.00.

Pairwise sequence alignment is an alignment between any two given sequences. When more than two sequences are aligned, it is called a multiple sequence alignment. Pairwise sequence alignment can be further classified as local and global sequence alignment. Local sequence alignment finds the best approximate sub-sequence match within two given sequences. Local sequence alignments are designed basically to search for highly similar regions within the two given sequences. Some of the popular local sequence alignment algorithms are Smith-Waterman [1], FASTA [2], BLAST [3], Gapped BLAST [4], BLAT [5], BLASTZ [6], and PatternHunter [7]. Global sequence alignment, on the other hand, is designed to find the best alignment of both sequences in their entirety. Thus, global sequence alignment looks for global mapping between entire sequences. Some of the popular global sequence alignment algorithms are Needleman-Wunch [8], MUMmer [9], GLASS [10], AVID [11], and LAGAN [12].

1.3 Structure of this survey

This paper focuses on pairwise sequence alignment. We begin the survey with optimal alignment algorithms for both local and global pairwise sequence alignment. Heuristic algorithms are covered in the later section and presented in two groups: local and global.

2. DEFINITIONS AND TERMINOLOGY

In this section, we introduce several fundamental definitions that are necessary to understand the algorithms described later.

Definition 1: Let L be a set of characters called alphabet set. A sequence S is an array of characters from L , such that they are all written contiguously from left to right and occupy a unique position in the sequence S . $|S|$ denotes the length of S and $S[i]$ denotes the i^{th} character of S .

In order to quantify an alignment, a scoring function is used. First, scoring function for the alignment of pairs of characters is defined and then scores of aligned pairs of characters are added to get the sequence alignment score. Based on the relationship between pair of characters in an alignment, the alignment score function σ can be classified into three types.

Definition 2: A character alignment score σ is a real valued function on pairs of characters. The score function σ is called:

- Match alignment score if the characters are same
- Mismatch alignment score if the characters are not indels and also different.
- Gap alignment score if one character is indel and the other is not.

Definition 3: Let S_1 and S_2 , be two sequences of length n . The alignment score ρ of S_1 and S_2 is defined,

$$\rho(S_1, S_2) = \sum_{i=1}^n \rho(S_1[i], S_2[i]).$$

The sequence alignment scoring function ρ defined above is called sum-of-pair (SP).

2.1 Scoring function

We now discuss some well known scoring functions. The simplest is the constant function [13]. For any given pair (x, y) , $x \neq y$,

$$\sigma(x, x) = a, a \in \mathbb{R}^+$$

$$\sigma(x, y) = b, b \in \mathbb{R}^-$$

$$\sigma(x, -) = \sigma(-, x) = c, c \in \mathbb{R}^-$$

The constant values a , b and c are usually obtained from a scoring matrix. A scoring matrix is an $n \times n$ matrix in which each cell contains a score for the corresponding pair of bases [13]. Scoring matrices are derived using statistical methods based on biological observations and historical sequence data. PAM (Percentage of Acceptable point Mutations per 108 years) series of matrices [14] [15] and BLOSUM (BLOCKS SUBstitution Matrix) series of matrices [16] are popular scoring matrices.

2.2 Gap alignment scoring function

The gap alignment score value is referred to as *gap penalty*. There are two types of gap alignment scoring functions namely, constant gap scoring function and affine gap scoring function.

In a typical gap alignment scoring function, the gap penalty score is fixed irrespective of the location in the aligned sequences. This is called *constant of fixed gap penalty score function*. In practice, mutations usually occur as a block of contiguous columns with a gap in the same sequence. This observation supports the evolutionary model, that is, given S and T , an ancestral sequence, U , then there is a probability that a block of contiguous columns of gaps occurred in course of time, separating U from S and T . In order to score such events of long contiguous block of gaps, a score *gap open penalty* is given at the beginning of each gap, and for each subsequent gap in the same block, a *gap extension penalty* is given. Such scoring function, where the penalty is different for the first and subsequent gaps, is called *affine gap penalty*. It can be defined as follows.

Definition 4: Affine gap score function has two components: open gap penalty d and gap extension penalty e . Now the affine gap penalty score can be computed as $d + l * e$, where l is the length of the gap.

2.3 Performance metrics

Let S and T be two sequences. The objective is to find an alignment of S and T , which has the maximum possible score for these two sequences [17]. The algorithms proposed in literature use the following metrics to determine their impact:

- Space and time efficiency.
- Sensitivity.

The sensitivity is measured by four different scores: percent identity score [17], total column score (TCS) [9], percent similarity score [17] and maximum alignment score. The two types of alignment scores are: total alignment score (that is the alignment of the whole sequence) and score of a filtered region of final alignment [11].

3. OPTIMAL ALIGNMENT ALGORITHMS

Over the past several decades, many algorithms have been proposed for both local and global sequence alignment. These algorithms fall under two distinct categories: optimal and heuristic. Optimal algorithms focus to produce the best alignment while heuristic algorithms produce near-best alignment.

A popular optimal local sequence alignment algorithm is the Smith-Waterman algorithm [1] proposed in 1981 and is based on a technique called *dynamic programming* [18]. Let S and T , be two sequences of length m and n , respectively. Let σ be character alignment score function. *“The dynamic programming algorithm builds up the optimal score of an alignment between S and T by computing the optimal scores of alignments between all characters*

of S and T . Let $V(i, j)$ be the value of the optimal alignment of strings $S[1]...S[i]$ and $T[1]...T[j]$ [1]. The algorithm finds optimal values, $V(i, j)$ with $0 \leq i \leq n$ and $0 \leq j \leq m$ [17]. A sequence scoring table with $m + 1$ columns and $n + 1$ rows is constructed. The first row and first column is filled with zeroes. The algorithm computes an optimal alignment between $S[1..i]$ and $T[1..j]$, recursively using the following formula.

$$V[i, j] = \max \begin{cases} V[i-1, j] + \sigma(S[i-1], -) & i > 0; j \geq 0 \\ V[i-1, j-1] + \sigma(S[i], T[j]) & i, j > 0 \\ V[i, j-1] + \sigma(-, T[j-1]) & i \geq 0; j > 0 \end{cases}$$

The highest score in the matrix is the optimal score, $\text{opt}(S, T)$. During the matrix computation, information on each sub-optimal score $V(i, j)$ is obtained according to the above equation. On completion of the matrix, a procedure traces back from the highest score cell to the cell containing zeros (0, 0). The alignment is actually built during this trace back stage.

Needleman-Wunsch Algorithm [8] is an optimal algorithm for global sequence alignment. The algorithm uses the same formula as the local sequence alignment. The difference in the algorithm is that, upon completion, last row-last column in the matrix hold the optimal score. A similar trace back procedure is employed to construct the global alignment.

4. HEURISTIC ALGORITHMS

Heuristic algorithms differ from optimal alignments in that they do not find the optimal alignment but find an alignment closer to the optimal alignment, but much quicker. Heuristic algorithms are based on the observation that there are conserved regions in both sequences which have high alignment score. If these conserved regions are identified in advance, a local alignment can be built around them.

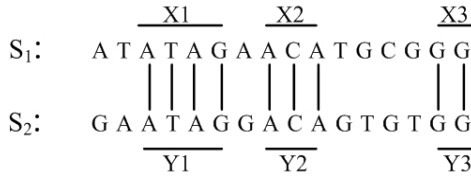


Figure 2: Conserved regions

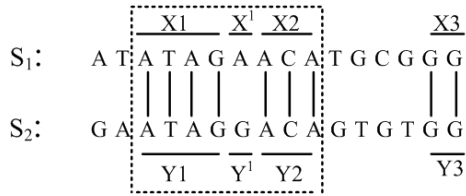


Figure 3: Highly similar regions in the two sequences

4.1 Characterizing the conserved regions

Consider the sequences S_1 and S_2 as shown in Figure 2. In this example, we see that there are regions in S_1 which are clearly aligned with regions in S_2 . On the other hand, in Figure 3, it is logical to assume that $X1X'X2$ and $Y1Y'Y2$ are conserved regions with a mutation at either X' or Y' . If such highly similar regions could be identified, then they can be aligned with each

other easily. Such pair of highly similar regions is referred to as 'seed'. High similarity between $X1$ and $X2$ could be

1. HS1: A perfect match, every character in $X1$ matches the character in the corresponding position in $Y1$.
2. HS2: $\rho(X1, X2) \geq t$, for a given t and care positions are not fixed.
3. HS3: $X1$ matches closely with $X2$, such that care (and do not care) positions are fixed unlike the other seeds already mentioned.
 - a. HS3:1 K mismatches in do not care positions and m - k matches in care positions [BLAT seeds].
 - b. HS3:2 K -mismatches in k do not care positions [Spaced seeds].
 - c. HS3:3 The score of $\rho(X1, X2)$ in care position of Xp , $Xp+1...Xp+t-1$ and Yp , $Yp+1...Yp+s-1 \geq t$, for a given t [Vector seeds].

Definition 5: Given a threshold value 't' and a scoring matrix, $X1$ and $X2$ are considered a seed, if there exists a contiguous region $X2$ in S_2 , such that, the alignment score ρ of $X1$ and $X2$ defined by,

$$\rho(X1, X2) = \sum_{i=1}^n \sigma(X1[i], X2[i]) \geq t$$

In the example shown in Figure 4, $X1$ is PQG (size = 3), we see two such possible combinations of $X2$, when aligned with $X1$, their alignment score exceeds or is equal to threshold value 13. All such combinations of $X1$ and $X2$ can be considered as seeds.

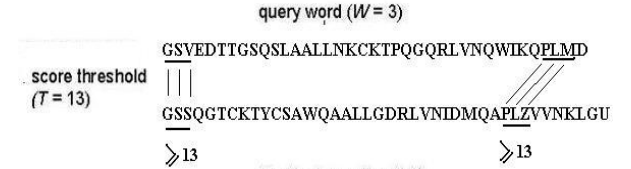


Figure 4: Seeds above a threshold score

Definition 6: $X1$ and $X2$ of equal length m are considered as a near perfect match seed or BLAT seed, if there exists a contiguous region $X2$ in S_2 , such that there are only r mismatches and the position where a mismatch is allowed is fixed.

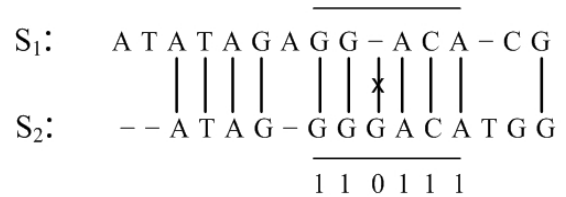


Figure 5: BLAT seed with one mismatch

In Figure 5, we observe a seed with one mismatch allowed in the third position. If a seed exists in S_1 and S_2 such that, the position where mismatch is allowed is actually a match, then it is also considered as a seed. The seed is characterized by ones and zeros, ones representing the position where there should be a match and zeros where a mismatch could be allowed. Thus, the above seed can be represented as 110111.

Definition 7: X_1 and X_2 of equal length m are considered as a spaced seed, if there exists a contiguous region X_2 in S_2 , such that there is minimum number of mismatches and the position where mismatches are allowed need NOT be fixed.

(1 1 1 0 1 1 0 0 1 0 1 0 1 1 0 1 1, 12)

Figure 6: Spaced seed with weight 12

In Figure 6, a spaced seed [7] of length 18 is given. The seed is represented by ones and zeros. The ones representing the position where there should be a match and zeros where mismatches could be allowed. The last number 12 represents the seeds weight (number of ones in the seed).

Let the alignment between two sequences $Y=y_1, y_2, \dots, y_n$ and $Z=z_1, z_2, \dots, z_n$ be a sequence of positional scores, $(s_{y_1, z_1}, s_{y_2, z_2}, \dots, s_{y_n, z_n})$, where $S=(s_{i,j})$ is the scoring matrix, then vector seed [19] is defined as follows.

Definition 8: A vector seed is an ordered pair $Q = (v, T)$, where v is the seed vector $(v_1, v_2, \dots, v_l)$ of real numbers and T is the seed threshold value. $X_1 = (x_1, x_2, \dots, x_n)$ hits the seed Q at position p if

$$\sum_{i=1}^l (v_i \cdot x_{p+i-1}) \geq T.$$

That is, the dot product of the seed vector and the alignment sequence of length ' l ' beginning at position p is at least the threshold T . The number of nonzero positions in the vector v is the support of the seed.

Figure 7 shows a general classification of seeds based on the above definitions.

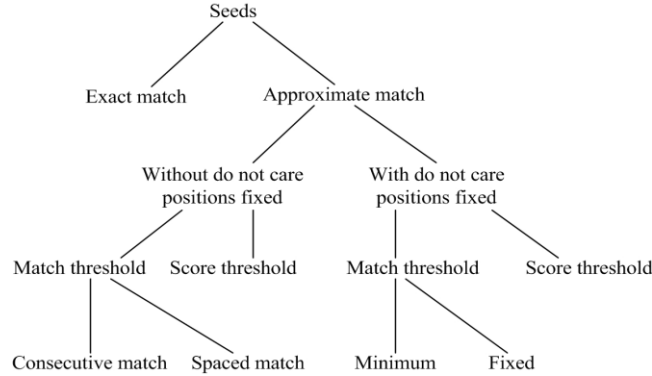


Figure 7: Classification of seeds

4.2 Seed/Anchor searching

There are two popular seed/anchor searching techniques, look-up table and suffix tree structure. In this section, we are going to briefly explain how these two techniques differ and apply to alignment process.

4.2.1 Look-up table

The look-up table is a data structure, usually an array, used widely in heuristic local sequence alignment problems. The main idea of the look-up table is to have seeds occupy unique positions in an array such that the position of seed in the array is obtained from the value computed using a hashing function. Once the look-up table is established for S_1 , one can linearly move through S_2 and

find all seeds of length ' l ' in S_1 using the look-up table. The positions of seeds in sequence S_2 is known when moving across it. In this way, seeds can be found in time $O(m)$, where m is the length of S_2 .

4.2.2 Suffix tree

Suffix tree is a data structure that represents the "internal structure of a string in a comprehensive manner" [20]. The exact matching problem can be solved in linear time $O(n)$, where n is the length of the string. Weiner developed the first linear time suffix tree back in 1973 [21], which was later improved by [22] [23]. An example of suffix tree is shown in Figure 8.

Definition 9: A suffix tree for an m -character string S is a rooted tree with exactly m leaves numbered 1 to m . Each internal node, other than the root, has at least two children and each edge is labeled with a non-empty substring of S . No two edges out of a node can have edge labels beginning with the same character [20]. The key feature of the suffix tree is that for any leaf i , the concatenation of the edge-labels on the path from the root to leaf i exactly spell out the suffix of S that starts at position i . That is, t spells out $S[i..m]$.

A generalized suffix tree is a suffix tree that combines the suffixes of the string $S_1, S_2, \dots, S_n$. An improvement in terms of speed and memory efficiency was proposed by [23], also known as online construction suffix tree algorithm. The two popular applications of suffix tree are: solving longest common substring problem and maximal repeat string problem in the sequence [20].

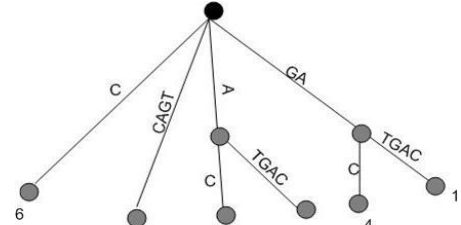


Figure 8: Suffix tree for the string GATGAC

4.3 Local sequence alignment algorithms

4.3.1 FASTA

FASTA is the first algorithm to use seeds and a look-up table for indexing the first sequence [20]. The algorithm starts by identifying perfect match seeds of length ' k ' in both sequences using the look-up table. FASTA uses a 'diagonal' method to find all seeds along a diagonal between the two sequences. All seeds are scored giving a positive score for the seeds and negative score for the intermediate regions. Thus, groups of seeds with high scores contribute more to the local diagonal score than seeds with low similarity scores. In the process, there could be ' n ' number of diagonal seeds, of which, FASTA saves the 10 best seeds. FASTA identifies a diagonal, which scores the highest value, 'init1' and identifies 'good' diagonals. A good diagonal is one, which has a score above a threshold [24]. All such diagonals are then combined into a single high scoring alignment allowing spaces by constructing a directed weighted graph around the seeds. The maximum weighted graph is then selected and the best alignment found is marked as 'initn'. In the next stage, FASTA builds a narrow band of width ' k ' centered along init1 [2]. FASTA then computes an optimal local alignment in this band by using Smith-

Waterman algorithm assuming that the optimal alignment lies within this band.

One of the disadvantages of FASTA is that, if the sequences under consideration have more than one region of homology (two optimal diagonals), only region around *init1* is found while the region, which contributed to *initn*, is discarded. Seeds, which are less than ‘k’, could be missed. The main advantage is the speed over optimal algorithm.

4.3.2 BLAST

BLAST (Basic Local Alignment Search Tool) uses a look-up table to identify seeds and considers seed which have a score above a threshold ‘t’ for protein sequence alignment, and an exact match seed for DNA alignment. The default length of the seed is 3 for protein sequences and 11 for DNA sequences. We consider protein sequence for simplicity in this survey.

For each seed in the sequence, a set of neighbourhood words, which exceed threshold ‘t’, is generated. The set of neighbouring words as well as the exact matches for the seed are then used to match against the second sequence. After finding seeds, BLAST algorithm extends the seed alignment in both directions without introducing any gaps. When the alignment is extended on either side, an alignment score can increase or decrease. If the alignment score drops below a predefined threshold, the extension is stopped. After extending the seed alignment in both the directions, if the score is above a certain threshold, then such seed segment pair is called high scoring pair (HSP). BLAST now extends with gaps around the HSP, using Smith-Waterman algorithm until the score falls below a threshold ‘r’. The best HSP is taken and reported as an alignment.

The disadvantages of this algorithm are that it cannot find seeds smaller than the minimum length ‘l’ considered for the exact match seed (DNA alignment) and reports only local alignments. It also finds too many seeds per sequence thus reducing speed (protein alignment) and does not allow for gaps in sequence. To overcome these disadvantages, BLAST2 [4] was developed.

4.3.3 BLAT

BLAT (BLAST Like Alignment Tool) [14] algorithm is similar to BLAST and FASTA as it searches for seeds of fixed length ‘l’ using look-up table, and the final alignment is built around the seeds found. BLAT differs from BLAST on which sequence it is indexing. BLAT builds an index of “non-overlapping seeds of S2 database sequence and scans linearly through S1, whereas BLAST builds an index of the S1 and then scans linearly through the database” [5]. BLAT finds both perfect match seeds and BLAT seeds using look-up table and extends them in both directions. After this initial extension, BLAT stitches them together to form a larger alignment.

The BLAT algorithm first searches for perfect match seeds of length ‘k’, and consider perfect match seeds, which are within a distance ‘d’. The seeds considered for indexing are all non-overlapping seeds. Sequence S1 is searched linearly from left to right considering overlapping seeds. BLAT now searches for near perfect match seed in both the sequences using a look-up table. When multiple seeds are within a distance, ‘d’, they are extended to form a single seed [5]. All seeds found are extended into high scoring pairs (HSPs), very similar to the HSPs found in the BLAST. Stitching the HSP’s is similar to the band created around the best diagonal discussed in FASTA algorithm with the only difference being there is no band in BLAT. In order to stitch the

seeds, two adjoining seeds should be within a distance ‘d’. BLAT later reports this as alignment. BLAT continues to be limited in the sense that it cannot find small homologous regions because of the small seed length considered.

4.3.4 PatternHunter

PatternHunter [7] introduced spaced seed to further improve the sensitivity and speed achieved by BLAST and BLAT. PatternHunter uses priority queues, a variation of red-black tree, queues, and hash table to achieve its speed [25]. PatternHunter first builds an index of S1 for the spaced seed model considered (e.g., 1110010011, with 6 care and 4 do not care positions in a 10-tuple seed length), similar to BLAST. Using the look-up table, the first position where the model fits in the second sequence is recorded; this is considered as a hit. Subsequent hits or positions where the model fits is recorded in another table, ‘hit table’. For each hit, hits along its diagonal are considered and hits which are overlapping or to the right are ignored. The hits are extended to the right and left until the score of the segment falls below a threshold value ‘t’. All such segment pairs, which have a score above the threshold are considered as high scoring pair (HSP). These HSP’s are then extended to the left. Initially, all HSP’s are diagonally sorted using a variation of red-black tree. Seeds of a smaller model 1101(spaced seed of length 4 with 3 care positions and do not care position) in a limited length ‘l’ from the left of the HSP are also sorted along with other HSPs. Next, PatternHunter connects HSP’s which are separated by a distance, and those which are overlapping are penalized with a score (are shrunk to fit). This process is repeated to find the best HSP. In the end, the algorithm computes the best partial alignment score.

PatternHunter2 uses multiple seed design or model instead of only one as in PatternHunter. In two hit mode, a “gapped extension is performed only if two nearby hits are found on the same diagonal” [6].

The algorithm works better than the algorithms discussed earlier in terms of sensitivity. The speed of the algorithm is not better than BLAST as it is implemented in Java and incurs memory problems for very long sequences.

4.3.5 BLASTZ

BLASTZ [6], is the fastest algorithm in the BLAST series. In order to speed up the algorithm, all repeats in the sequences are removed. A repeat is a substring of same length repeated along the length of the sequence. BLASTZ initially looks for all pairs of identical seeds of length ‘k’, except for at most one transition. A transition is shift from one character to other. For example, in a DNA sequence, a transition is A-G, G-A, C-T or T-C. Look-up table is again to record the matches in both the sequences. Each seed is extended in both directions without gaps. The extension is stopped when the score drops below some threshold X. All the segments after the gapped alignment which score above, say Y, are retained. Let us call all such segments as ‘zones’. For the regions between the zones, BLASTZ repeats the above procedure using a more sensitive seeding procedure (e.g., 7-mer exact matches) and lower score thresholds, say X-d for gap free threshold and Y-e for gapped extension. BLASTZ finally adjusts the sequence positioning such that all masked segments could now also be included in the final alignment.

Although BLASTZ uses transitions seeds, the length of the seed being 12 is too long to find small regions of homology in divergent sequences. Hence BLASTZ fares well with naturally

evolving sequences but fails relatively poorly with divergent sequences. Due to the seed length being 12, and only one transition allowed in the entire seed length, BLASTZ finds too many seeds, and thus spends most of the time in calculating HSP's.

4.3.6 MASAA

MASAA (Multiple anchor staged alignment algorithm) [26] uses the searching techniques (suffix tree) used in global sequence alignment algorithms to find long common substrings in both the sequences. The final alignment is built around these long common substrings. These substrings could also be considered as long perfect match seeds. The algorithm is based on an intuition that long substrings would more or less be part of the final alignment. The algorithm has five phases.

In the first two phases, all large substrings which are at least 50 percent of the longest substring in both the sequence are found. These substrings are all non-overlapping and are consistent (Figure 9) in both the sequences. In the next two stages, the regions in between the long substrings found earlier are again searched for smaller seeds (use BLAT seeds of length 12 with 4 mismatches). The seeds considered in these regions could be overlapping and criss-crossing. In the last stage, the alignment is triggered from the longest substring found in the first two stages, and if there is a substring within a distance 'd' from it, extension on both sides continues until there are either no substrings within a distance 'd' or there are no substrings at all.

The simulations illustrate that this algorithm outperforms BLASTZ when the sequences are divergent and, in some cases, produces an alignment when BLASTZ returns no alignment. On homologous sequences, the performance is comparable. While BLASTZ takes more time in searching for HSP's, MASAA takes time in the regions between long substrings; overall, MASAA takes less time than BLASTZ to find the alignment.

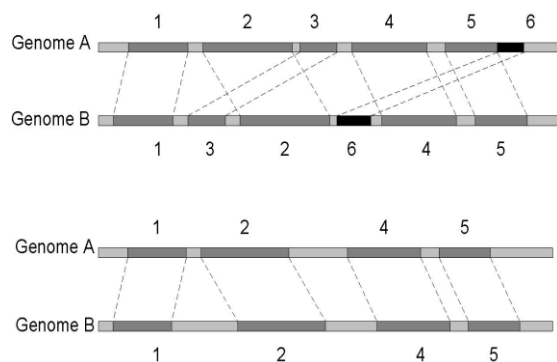


Figure 9: Criss-crossing and overlapping seeds. The bottom figure shows consistent seeds.

4.4 Global sequence alignment algorithms

4.4.1 MUMmer

MUMmer (Maximal Unique Match-mer) [9], was one of the first global alignment algorithms designed to align two long genomes. MUMmer makes use of suffix tree, longest increasing subsequence (LIS) and Smith-Waterman alignment for computation time and memory efficiency. A term 'SNP', meaning,

a subsequence which appears in both the sequences but with a difference of only one base is introduced. The algorithm has five stages: 1) identify SNP's, 2) identify segments where two genomes differ by more than one SNP, 3) identify large segments which appear only in one genome, that is, large insertion/deletion, 4) identify repeats – same substrings found more than once in one genome and not in the other, and 5) identify 'tandem repeats' - same substring which has been found different number of times in both the genomes. Given two sequences, a suffix tree is built for the two sequences. Finding maximal unique match takes one traversal from the root to all leaf nodes, that is, it can be found in $O(n)$, where n is the length of the match. The length of unique match MUMmer considered is at least half of the length of the longest maximal unique matches (MUM) found. MUMmer sorts matches which are consistent, that is, matches occur in the same order in both sequences using a variation of the LIS algorithm [20]. MUMmer uses Smith-Waterman algorithm between MUMs. Once the gaps between MUMs are aligned, an overall global alignment is established. In the regions between the MUMs, "MUMmer uses several algorithms for closing the local gaps and completes the final alignment" [9]. MUMmer heavily depends on the unique matches it finds in the first step, if MUMs are few, the algorithm performs poorly. MUMmer requires that two input sequences be homologous. When aligning heterogenous sequences, if the MUMs are spread wide apart, the inter MUM would have to be aligned with Smith-Waterman algorithm, thus increasing the computation time.

```

Genome A:  GGATTTGGATATCTGATCTTGAGGATAGGGATA
Genome B:  CCATTTGGATATTCTCTATTGAGGATAGGGCCC

Genome A:  GG%CTGATC#ATA
Genome B:  CC%TCTCTA#CCC

```

Figure 10: Converting a k-mer to a unique character after hashing the k-mer

4.4.2 GLASS

GLASS [10] was developed to align long genomic sequences. The goal was to overcome the limitations of standard dynamic programming (SDP) methods which had their running time of $O(nm)$ (where n and m are the lengths of the genomic sequences compared) and to improve sensitivity when aligning the sequences in their entirety. GLASS first finds all common K-mers in both sequences. The algorithm converts each matching K-mer to a unique character in both sequences. An example is shown in Figure 10. GLASS applies standard dynamic programming algorithm to regions (12 bps) on both ends of each K-mer and computes two scores which are added for each K-mer. GLASS considers only 'consistent' K-mers whose score exceeds a threshold T . Two K-mers are inconsistent if they are overlapping or criss-crossing. GLASS recursively aligns intervening regions using a smaller value of K . The value of K is 20, 15, 12, 9, 8, 7, 6 and 5. Once recursive procedure is performed, "GLASS extends all pairs of aligned segments by short local alignments to the left and right by SDP. Finally, GLASS aligns the remaining (usually short) unaligned regions using SDP" [10].

GLASS recursively aligns the two sequences hence, it is slower than MUMmer. However, as it considers short K-mers it is more sensitive than MUMmer. GLASS neglects k-mers of size shorter than what it is considered in the algorithm. The removal of overlapping and crossing seeds makes the algorithm less efficient for detecting trans-positions and reverse seeds.

4.4.3 AVID

AVID [11] attempts to balance both speed and sensitivity when aligning very long sequences. To achieve better computational efficiency, it uses suffix tree and considers overlapping anchors. To improve sensitivity, it uses a variant of Smith-Waterman algorithm, in the inter-anchor region. The algorithm starts by concatenating two sequences by placing a special character N between them. All maximal repeat substrings (longest sub-string which is repeated in both the sequences) are found by browsing all nodes in the suffix tree only once [27]. Thus, time taken is $O(n)$, where n is the length of the concatenated string.

AVID considers sub-string matches that are at least half the length of maximal repeat substring found. AVID then sorts the matches, which are non criss-crossing and non-overlapping, similar to GLASS and MUMmer. A variant of the Smith-Waterman algorithm is used to select anchors from those found in the previous stage. Every match is evaluated based on its length and alignment scores of its two regions (10 bp on each side). This is similar to the idea first employed in the GLASS algorithm. AVID picks only those anchors which score above a threshold. Shorter matches and repeat matches removed in the earlier stage are later considered for inter-anchor regions. The procedure is repeated recursively until there are no anchors found between two regions. For short regions, Needleman-Wunsch algorithm is used to get the final global alignment.

AVID is not sensitive when aligning distantly related sequences. If two anchors are relatively far away, AVID would end up using optimal algorithm thus increasing its computational time.

4.4.4 LAGAN

LAGAN [28] is more sensitive than previous pair-wise global sequence alignment algorithms discussed in this paper. LAGAN is an efficient and reliable pair-wise aligner that is suitable for genomic comparison of distantly related organisms. CHAOS algorithm [12] is used first to find local alignments between two sequences. The advantage of using CHAOS algorithm is that it “finds local alignments using multiple short inexact words instead of the longer exact words” [28] used by MUMmer and GLASS. “Given a maximum distance d and maximum range ‘ s ’, two local alignments or anchor x and y in the two sequences, can be chained together if the indices of x (starting address) in both sequences are higher than the indices of y , and x and y are ‘near’ each other” [28], with ‘near’ defined by both a distance and a gap criteria. The chaining is a global alignment between x and y . “The final score of a chain is the total number of matching bp in it. The default parameters used by CHAOS are words of length 10, a distance and gap criteria of 20 and 5 bp respectively” [28]. LAGAN then constructs an optimal rough global map using sparse dynamic programming. The procedure is applied recursively to all anchors. In the end, it computes the optimal Needleman-Wunsch global alignment within the range ‘ r ’ from the anchors to get the final alignment.

LAGAN spends most of its time in chaining local alignments found in the first stage of the algorithm. As a result, it is not faster than MUMmer nor BLASTZ. LAGAN, does not consider

overlapping seeds, making the algorithm less sensitive in detecting trans-positions.

5. CONCLUSION

Pairwise sequence alignment problem has been an extremely active area of research. In the recent past, due to new discoveries in seed model and indexing techniques, there has been a flood of new algorithms. The algorithms can be classified as pairwise or multiple sequence alignment depending upon the number of sequences involved. Pairwise algorithms can be further classified as local or global, and optimal or heuristic. All pairwise algorithms vary on the three basic stages: indexing stage, identifying seeds/ anchors and the final stage. Some algorithms are more suited to homologous sequences whereas others target divergent sequences. The heuristic algorithms produce alignments faster at the cost of reduced sensitivity.

6. REFERENCES

- [1] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequence," *J. Molecular Biology*, vol. 147, pp. 195-197, 1981.
- [2] D. J. Lipman and W. R. Pearson, "Rapid and Sensitive Protein Similarity Searches," *Science*, vol. 227, pp. 1435-1441, 1985.
- [3] S. F. Itschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic Local Alignment Search Tool," *J. Molecular Biology*, vol. 215, pp. 403-410, 1990.
- [4] S. F. Altschul, et al., "Gapped BLAST and PSI-BLAST: a new generation of protein database search programs," *Nucleic Acids Res.*, vol. 25, pp. 3389-3402, Sep. 1997.
- [5] W. J. Kent, "BLAT—The BLAST-Like Alignment Tool," *Genome Research*, vol. 12, pp. 656-664, 2002.
- [6] S. Schwartz, et al., "Human–Mouse Alignments with BLASTZ," *Genome Research*, vol. 13, pp. 103-107, 2003.
- [7] B. Ma, J. Tromp, and M. Li, "Pattern-hunter: faster and more sensitive homology search," *Bioinformatics*, vol. 18, pp. 440-445, 2002.
- [8] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *J. Molecular Biology*, vol. 48, pp. 443-453, 1970.
- [9] A. L. Delcher, et al., "Alignment of whole genomes," *Nucl. Acids Research*, vol. 27, pp. 2369-2376, 1999.
- [10] L. Batzoglou, J. Pachter, B. Mesirov, B. Berger, and E. S. Lander, "Human and mouse gene structure: comparative analysis and application to exon prediction," in *RECOMB '00: Proc of the 4th Int'l Conference on Computational Molecular Biology*, 2000, pp. 46-53.
- [11] N. Bray, I. Dubchak, and L. Pachter, "AVID: A Global Alignment Program," *Genome Research*, vol. 13, pp. 97-102, 2003.
- [12] M. Brudno and B. Morgenstern, "Fast and sensitive alignment of large genomic sequences," in *Proc of IEEE Computer Science Bioinformatics Conference*, 2002, pp. 138-147.
- [13] Pittsburgh Supercomputing Center. (2007) [Online]. http://www.psc.edu/research/biomed/homologous/scoring_primer.html

- [14] D. J. States, W. Gish, and S. F. Altschul, "Improved Sensitivity of Nucleic Acid Database Search Using Application-Specific Scoring Matrices," *METHODS: A Companion to Methods in Enzymology*, vol. 3, no. 1, pp. 66-70, 1991.
- [15] M. O. Dayhoff, R. M. Schwartz, and B. C. Orcutt, "A Model of Evolutionary Change in Proteins," in *Atlas of Protein Sequence Structure Nat'l Biomedical Res.*, M. O. Dayhoff, Ed. 1978, ch. 5, pp. 345-352.
- [16] S. Henikof and J. G. Henikof, "Amino acid substitution matrices from protein blocks," in *Proc Natl Acad Sci*, 1992, pp. 10915-9.
- [17] M. Tompa, "Lecture notes on Biological Sequence Analysis," University of Washington, Seattle, Technical report, 2000.
- [18] S. Dreyfus, "Richard Bellman on the birth of Dynamic Programming," *Operation Research*, vol. 50, no. 1, pp. 48-51, 2002.
- [19] B. Brejova, D. G. Brown, and T. Vinar, "Vector seeds: an extension to spaced seeds," *J. of Computer and System Sciences*, vol. 70, no. 3, pp. 364-380, 2005.
- [20] D. Gusfield, *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.
- [21] P. Weiner, "Linear Pattern Matching Algorithms," in *Proc. 14th IEEE Annual Symp. on Switching and Automata Theory*, 1973, pp. 1-11.
- [22] E. M. McCreight, "A Space- Economical Suffix Tree Construction Algorithm," *J. ACM*, vol. 23, pp. 262-272, 1976.
- [23] E. Ukkonen, "On-Line Construction of Suffix Trees," *Algorithmica*, vol. 14, pp. 249-260, 1995.
- [24] A. Kibria, et al. (2008) EBI2 Can Support Portal Bioinformatics - FASTA similarity search - Introduction. [Online]. <http://www.ebi.ac.uk/2can/tutorials/nucleotide/fasta.html>
- [25] D. G. Brown, *A survey of seeding for sequence alignments. Bioinformatics Algorithms: Techniques and Applications*. J. Wiley and Sons, 2007.
- [26] W. Haque, A. Aravind, and B. Reddy, "An Efficient Algorithm for Local Sequence Alignment," in *Proc 30th Int'l IEEE EMBS Conference*, Vancouver, 2008.
- [27] D. S. Hirschberg, "A linear space algorithm for computing maximal common subsequences," *Comm. ACM*, vol. 18, pp. 341-343, 1975.
- [28] M. Brudno, et al., "LAGAN and Multi-LAGAN: Efficient tools for large-scale multiple alignment of genomic DNA," *Genome Res.*, vol. 13, pp. 721-731, 2003.